

What it costs to run one listener

Per active listener / year — every cost, four wallet setups, one picture

MUSIC STREAMED / YR

\$98.00

9,800 min @ \$0.01/min

PLATFORM RAKE · 10%

\$9.80

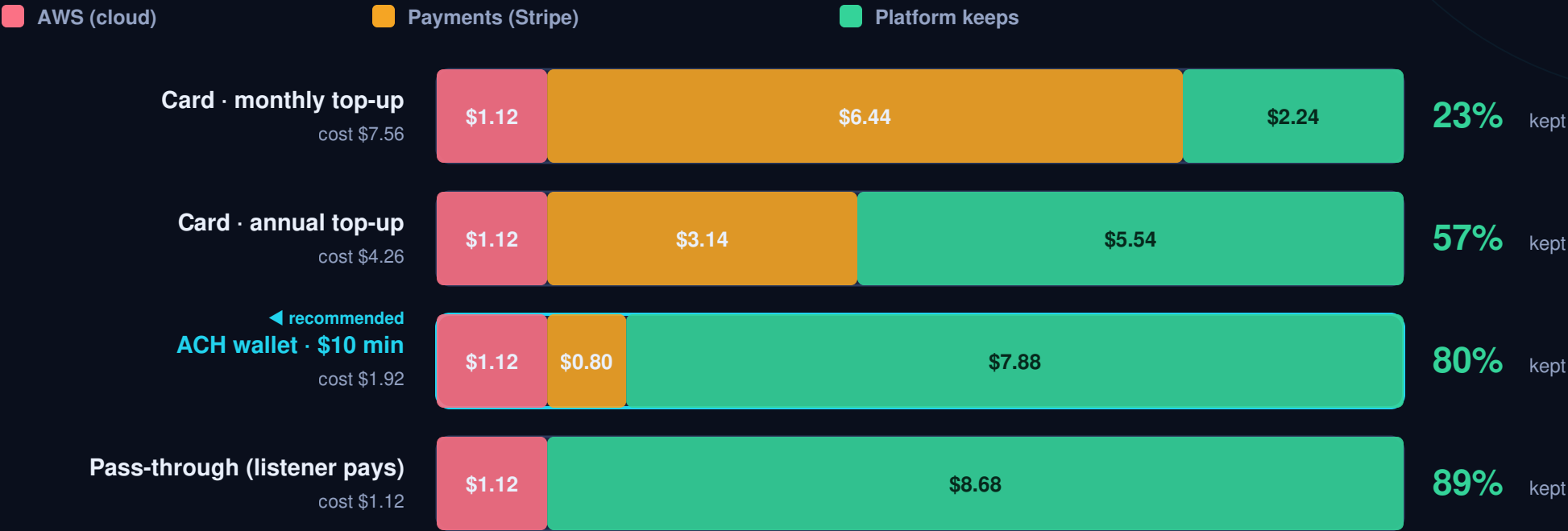
the platform's cut of that \$98

PLATFORM KEEPS · ACH

\$7.88

recommended wallet — 80% of rake

OF THE PLATFORM'S \$9.80 RAKE — WHAT INFRASTRUCTURE EATS, AND WHAT'S LEFT

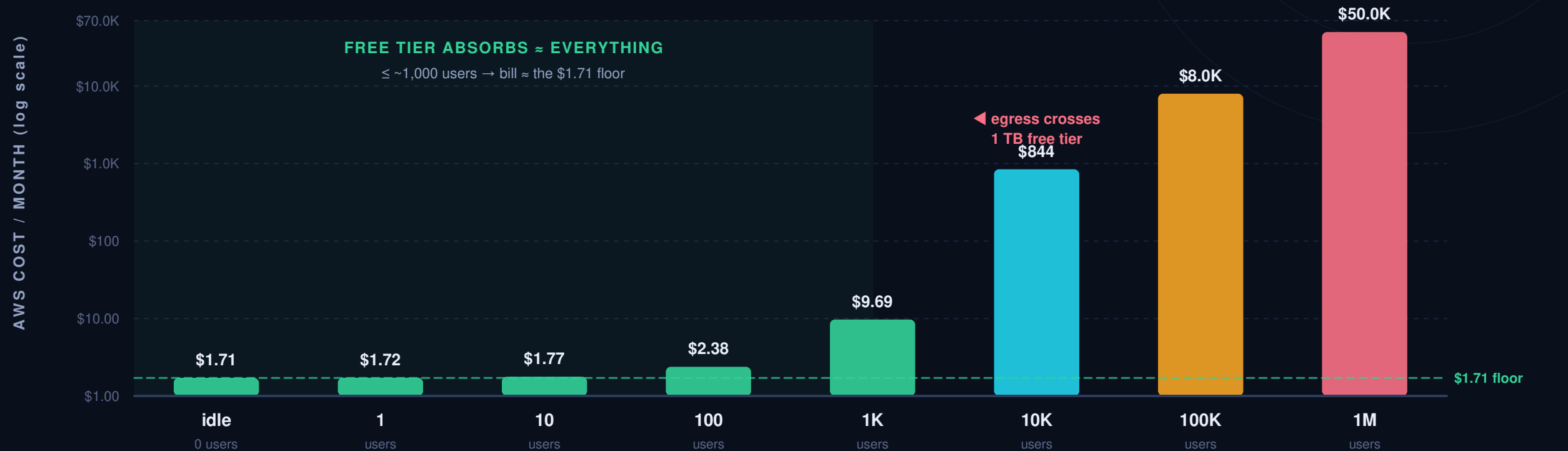


AWS is fixed at ~\$1.12/listener/yr — the payment setup is the whole lever, swinging the rake kept from 23% to 88%.
The artist's \$88.20 royalty (90%) is separate; its payout fee comes from the artist's share, not this rake.

Idle to one million users

Monthly AWS bill across eight scales — log scale, free tiers applied

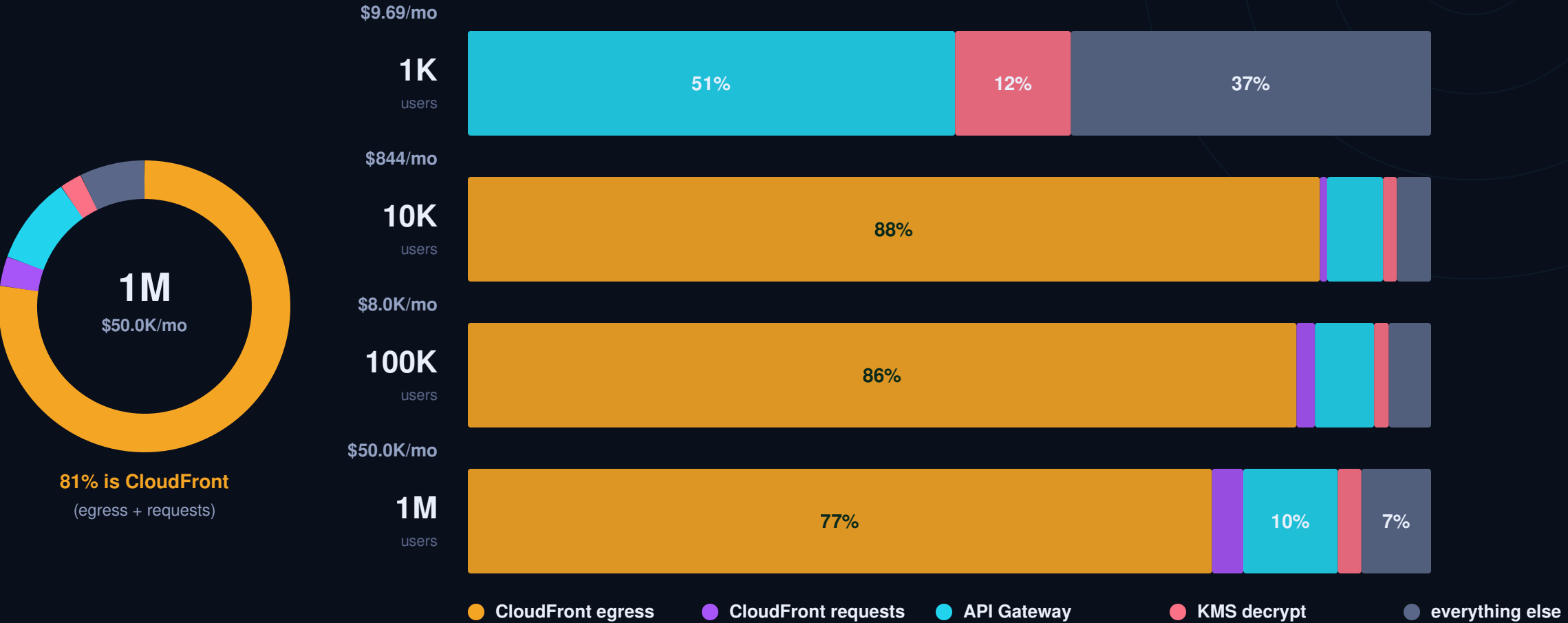
Monthly AWS bill scales with one thing — bytes out of CloudFront. Below ~1,000 users the always-free tiers swallow the whole stack; the bill is the \$1.71 idle floor. The first real step up is the egress cliff at ~10K users, after which volume tiers bend the curve back down.



Egress is the whole story

What each dollar of the AWS bill is actually spent on

CloudFront egress is the majority of every bill. After the API re-platform, API Gateway is the largest of the rest; DSQL, Lambda, DynamoDB, S3 and the KMS key stay small. TollRoad is a ba

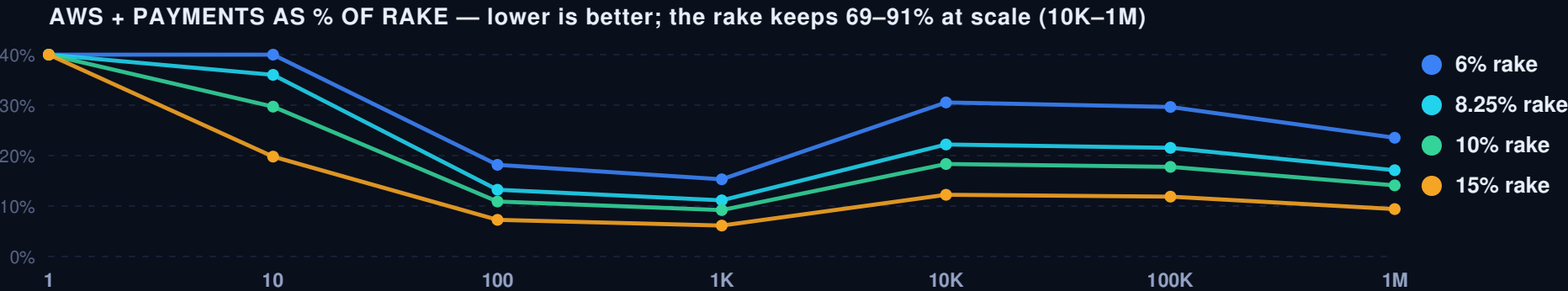


The rake covers AWS + payments

Platform profit and cost-as-%-of-rake (AWS + 0.8% ACH) across scales × four rake levels

	1 users	10 users	100 users	1K users	10K users	100K users	1M users
6% rake	-\$1.29 /mo profit	\$2.48 /mo profit	\$40.11 /mo profit	\$415 /mo profit	\$3.4K /mo profit	\$34.5K /mo profit	\$374.9K /mo profit
8.25% rake	-\$1.11 /mo profit	\$4.31 /mo profit	\$58.49 /mo profit	\$599 /mo profit	\$5.2K /mo profit	\$52.9K /mo profit	\$558.7K /mo profit
10% rake	-\$0.97 /mo profit	\$5.74 /mo profit	\$72.79 /mo profit	\$742 /mo profit	\$6.7K /mo profit	\$67.2K /mo profit	\$701.7K /mo profit
15% rake	-\$0.56 /mo profit	\$9.83 /mo profit	\$114 /mo profit	\$1.2K /mo profit	\$10.8K /mo profit	\$108.0K /mo profit	\$1.11M /mo profit

Cell = platform profit after AWS + payments (rake – AWS – 0.8% ACH). Red = underwater.

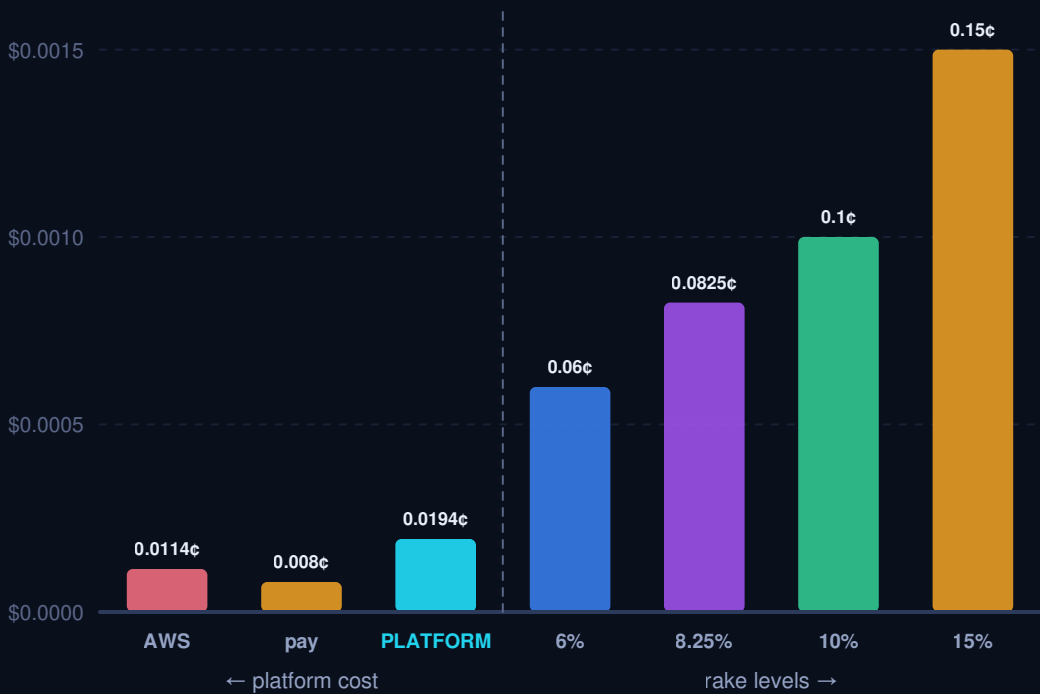


All-in: cost vs. rake, and the flow at scale

Every cost merged — the unit that matters, and where a million users' money goes

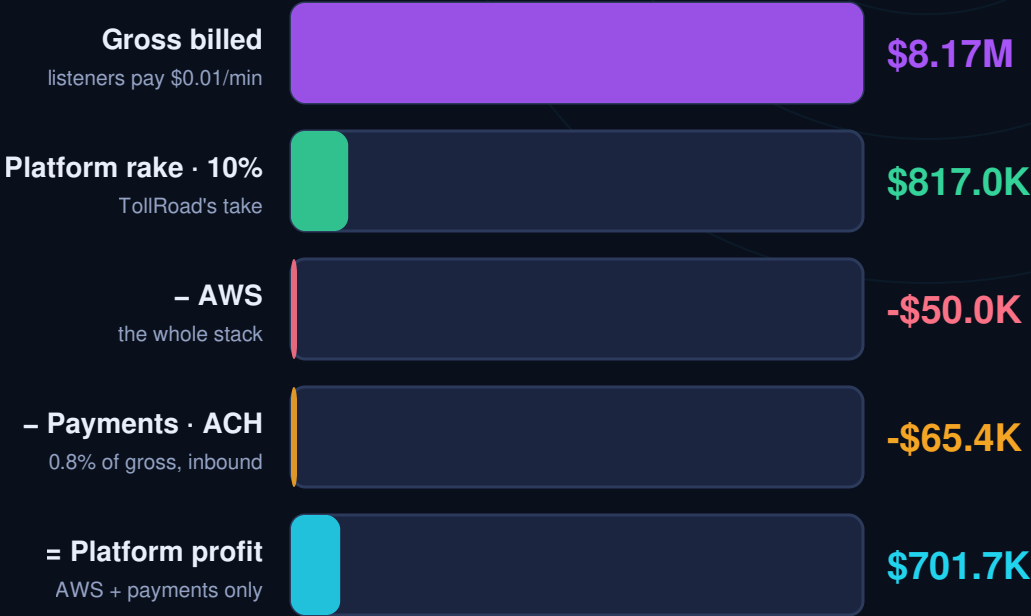
PER STREAMED MINUTE — all-in cost vs. rake

The three cost parts, their sum, and what each rake level takes.



Every rake level clears the platform cost — the 10% rake by ~5.1x.

ALL-IN FLOW AT 1,000,000 USERS / MONTH



Payments (\$65K) now outweigh AWS (\$50K). The platform keeps 86% of the rake here; the artist payout (~\$18K/mo) is drawn from the \$7.35M royalty, not this flow.

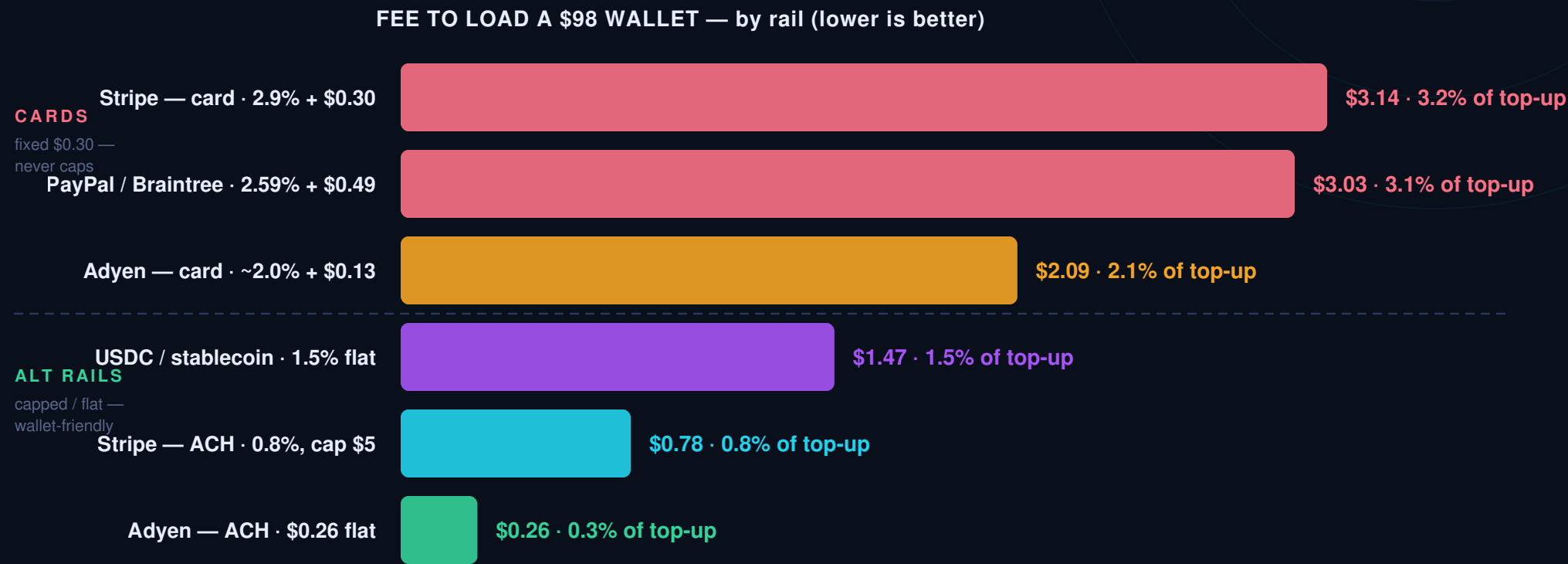
Annual platform profit @ 10% rake, 1M users:

\$8.42M / year

The rail is the real margin lever

Stripe vs. the alternatives — cost to top up a prepaid wallet

Pages 1–5 sized AWS at ~1% of gross. Card processing runs 3–7× that — payments, not compute, set the real margin floor. The lever is the rail: a card's fixed \$0.30 never amortizes, while bank rails (ACH) CAP the fee — ideal for a prepaid wallet that loads in large, infrequent chunks.



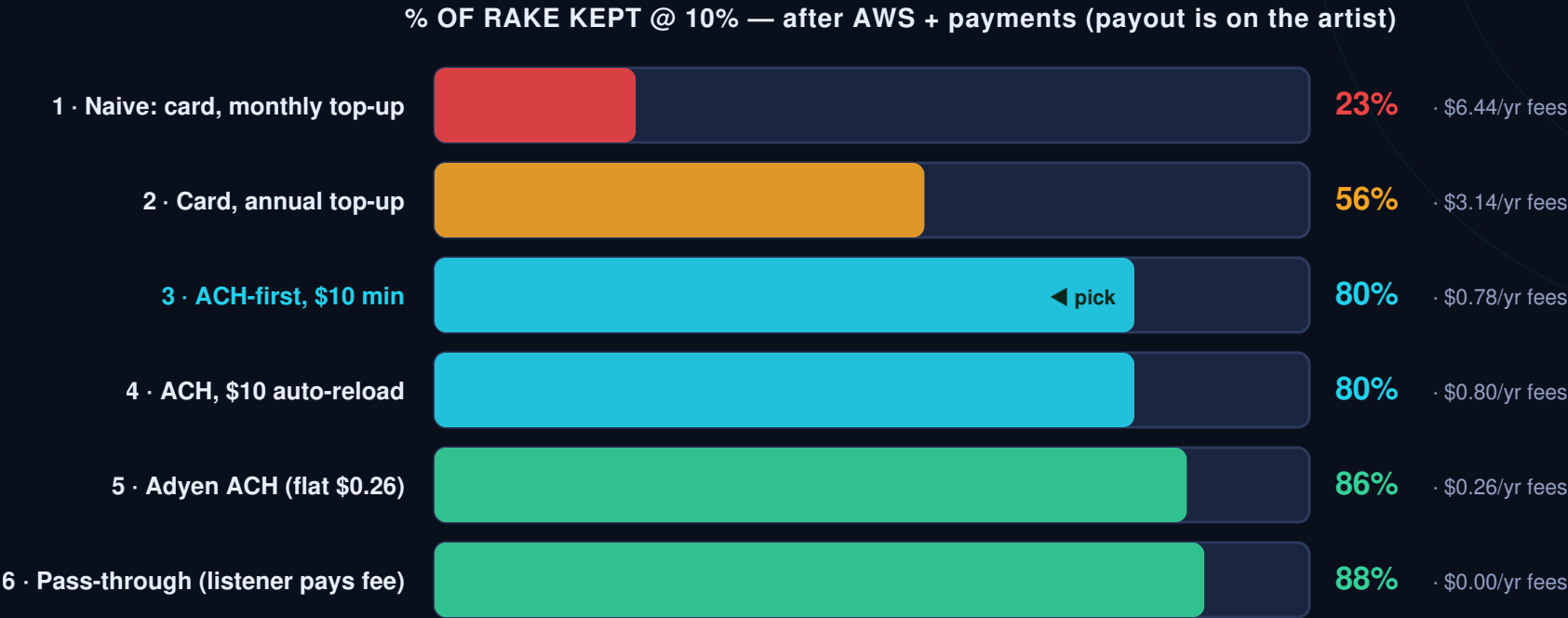
Cards never escape the \$0.30 fixed fee. ACH caps it — Stripe 0.8% to a \$5 ceiling, Adyen a \$0.26 flat — and USDC is a flat 1.5%.

A wallet loaded in big chunks barely feels a capped fee → make ACH the default and cards the instant fallback.

Six ways to run the top-up wallet

The configuration — not the cloud bill — sets the platform's take

Same wallet, six configurations. The choice of rail + top-up size + who eats the fee swings the platform's kept margin from 23% to 88% of the rake — a bigger lever than anything in the AWS stack.



RECOMMENDED — ACH-first wallet, \$10 minimum, card fallback

\$9.80 rake – \$1.12 AWS – \$0.80 ACH (inbound)

= \$7.88 kept (80% of the rake)

Pass-through → 88%. Artist payout is drawn from the royalty, not the rake.

THE FOUR KNOBS

- **Default rail:** ACH-first · card fallback
- **Minimum top-up:** \$10 — keeps even card < 6%
- **Fee handling:** platform absorbs (or pass-through)
- **Reload:** auto-reload at low balance